SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Grama Jyothi Billing Program

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Design, Implement and Validate a Reliable Billing Program
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

TASK 10 – Compare Expectations with Execution

Aim

To compare the expected behaviour of the type choices with measured results. Each bill = Rs 275.50 (27,550 paise).

Bills	float total (Rs)	Exact paise total (Rs)	float − exact	int paise total	long total (paise)
4,000	1,102,000.00	1,102,000.00	0.00	110,200,000	110,200,000
60,000	16,544,776.00	16,530,000.00	+14,776.00	1,653,000,000	1,653,000,000
800,000	218,557,600.00	220,400,000.00	−1,842,400.00	565,163,520 (overflow)	22,040,000,000

Observations: (1) the float total is exact at 4,000 only by luck (all intermediate sums are exactly representable); at 60,000 and 800,000 it drifts by Rs 14,776 and Rs 1.84 million; (2) int overflows at district scale – 22,040,000,000 > 2,147,483,647 – and wraps to 565,163,520 (strictly, signed overflow is undefined behaviour); (3) long and long long are correct, but long is only 4 bytes on Windows, so long long is the portable choice.

Standard guarantees: char ≥ 8 bits; short and int ≥ 16 bits; long ≥ 32 bits; long long ≥ 64 bits; float ≥ 6 decimal digits (FLT_DIG), double ≥ 10 (mine: 6 and 15). My system (1, 2, 4, 8, 8) meets or exceeds these minimums; the extra width of int (4 vs the minimum 2) and long (8 vs 4) is an implementation choice of the 64-bit Linux data model (LP64).

Alternative Concept – district total in double

A double has a 53-bit mantissa, so it represents integers exactly up to 2^53 ≈ 9.0×10^15. A district total of 2.2×10^10 paise is therefore represented exactly if every addend is a whole number of paise (measured: double total = 220,400,000.00 exactly). It loses accuracy only when summing fractional rupees (e.g. 275.50 is exact but 162.60 is not) or beyond 2^53. A long long never loses accuracy but overflows beyond 9.2×10^18. Time per addition: O(1) for both (my measure for 800,000 additions: long long 0.45 ms vs volatile float 1.95 ms at -O0, so integer was about 4× faster; the numbers will differ by machine). Space: 8 bytes each.

Result: Expectations confirmed: float drifts, int overflows, long long is exact.